29. 2-3-4 Trees

T.V. GEETHA

epgp books

Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we will discuss another type of balanced trees where non leaf nodes can have a maximum of 4 children.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the concept of 2-3-4 Trees
- To discuss insertion into 2-3-4 Trees
- To describe deletion from 2-3-4 Trees
- To discuss the advantages and disadvantages of 2-3-4 Trees

29.1 2-3-4 Trees

In a 2-3-4 tree, all leaf nodes are at the same level however data can appear in all

nodes. The 2, 3, and 4 in the name refer to how many **links** to child nodes can potentially be contained in a given node. For non-leaf nodes, three arrangements are possible:

- A node with only one data item always has two children
- A node with two data items always has three children
- A node with three data items always has four children.

In other words, for non-leaf nodes with at least one data item (a node will not exist with zero data items), the number of links may be 2, 3, or 4. Because a 2-3-4 tree can have nodes with up to four children, it is also called a multiway tree of order 4. Non-leaf nodes must/will always have one more child (link) than it has data items - Equivalently, if the number of child links is L and the number of data items is D, then L = D+1. An example of a 2-3-4 tree is given in Figure 29.1

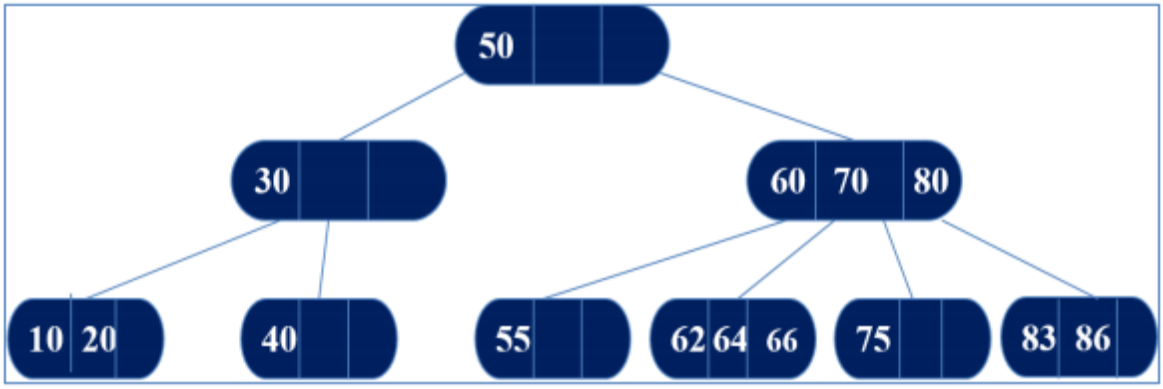


Figure 29.1 Example of a 2-3-4 Tree

Now let us look at the rules for placing data items in the nodes of a 2-3-4 tree.

- 2-node (two children), must contain a single data item that satisfies relationships
- 3-node (three children), must contain two data items that satisfies relationships
- 4-node (four children), must contain three data items that satisfies relationships

29.2 Features of 2-3-4 Trees

2-3-4 trees have some very nice features. They are always balanced. Recall the definition of a balanced tree that we discussed in the previous module. A *tree* is ***balanced*** (here unless otherwise specified balanced implies height balanced) if the difference in height between any of the node's subtrees is £ 1. The 2-3-4 tree is reasonably easy to program. It is also the basis for the understanding of B-Trees. B-Tree is another kind of multi-way tree particularly useful in organizing external storage, like files. B-Trees can have dozens or hundreds of children with hundreds of thousands of records.

Now let us go back to discussing 2-3-4 trees. As already discussed 4-node (four children) must contain three data items S, M, and L such that S is greater than left child's item(s) and less than middle-left child's item(s), M is greater than middle-left child's item(s) and less than middle-right child's item(s) and L is greater than middle-right child's item(s) and less than right child's item(s) (Figure 29.2). A leaf may contain either one, two, or three data items .

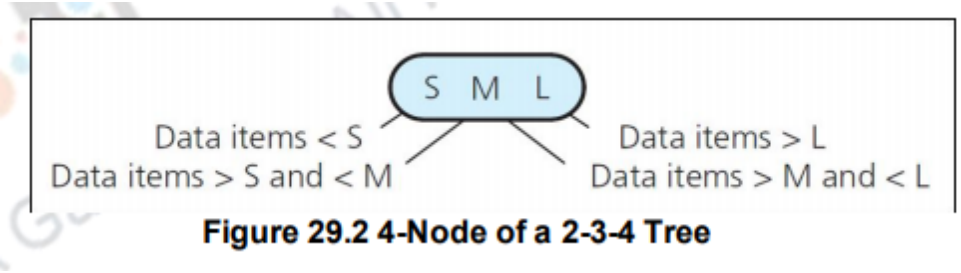


Figure 29.2 4-Node of a 2-3-4 Tree

Summarizing in a 2-3-4 tree, a 2-node has 1 value and a max of 2 children, a 3-node has 2 values and a max of 3 children and a a 4-node has 3 values and a max of 4 children. The numbers associated with the nodes refer to the maximum number of branches that can leave the node. same as a binary tree node (Figure 29.3).

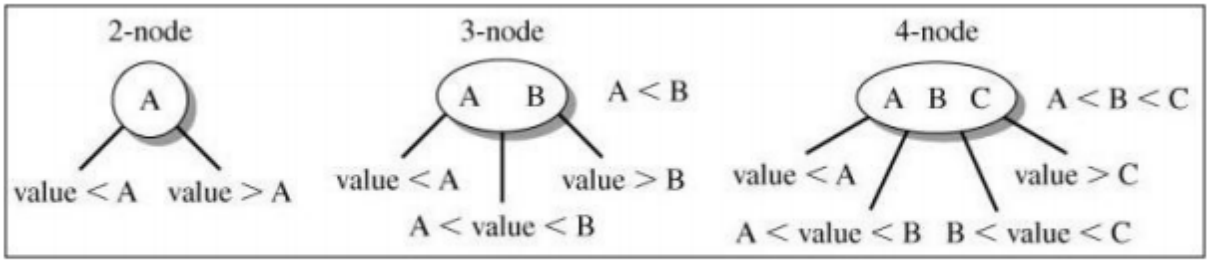


Figure 29.3 Nodes of a 2-3-4 Tree

The 2-3-4 trees must maintain height balance in all subtrees. This is called the *Depth property*. However the nodes are allowed to expand to accommodate inserts. In particular, nodes can have 2, 3 or 4 children, called as *Node-size property*, e.g., a 4-node would have 3 keys that splits the keys into 4 intervals. Searching and Traversing a 2-3-4 Tree are simple extensions of the corresponding algorithms for a 2-3 tree.

When inserting data into a 2-3-4 Tree, there may be splitting of nodes carried out by moving one of its items up to its parent node. There may be splitting of 4-nodes as soon as it encounters them on the way down the tree from the root to a leaf . In order to find an item, we start at the root and compare the item with all the values in the node; if there's no match, move down to the appropriate subtree and repeat until you find a match or reach an empty subtree

29.3 Insertion into 2-3-4 tree

For insertion to take place we first need to find the place for the item to be inserted as we do for any search tree. On the way down the tree, if you see any 4-nodes, split them and pass the middle value up. If the leaf is a 2-node or 3-node, add the item to the leaf. If the leaf is a 4-node, split it into 2 2-nodes, passing the middle value up to the parent node, then insert the item into the appropriate leaf node. There will be room, because 4-nodes were split on the way down.

If the leaf node into which the element is to be inserted is a 2 -node or 3-node, then simply insert the element into the leaf node.

If the leaf node into which the element is to be inserted is a 4-node, then this node splits and a backward (leaf-to-root) pass is initiated.

This backward pass terminates when either a 2-node or 3-node is encountered, or when the root is split.

To avoid the backward pass, we split 4-nodes on the way down the tree. As a result, the leaf node into which the insertion is guaranteed to be a 2- or 3-node.

There are three different cases to consider for a 4-node:

- It is the root of the 2-3-4 tree.
- It's parent is a 2 node
- It's parent is a 3 node

29.3.1 Splitting Nodes During Insertion

Let us discuss the splitting of a 4-node (Figure 29.4). In general as given in Figure 29.4 (1.) we split a 4-node having data values S, M and L with children a, b, c, d into

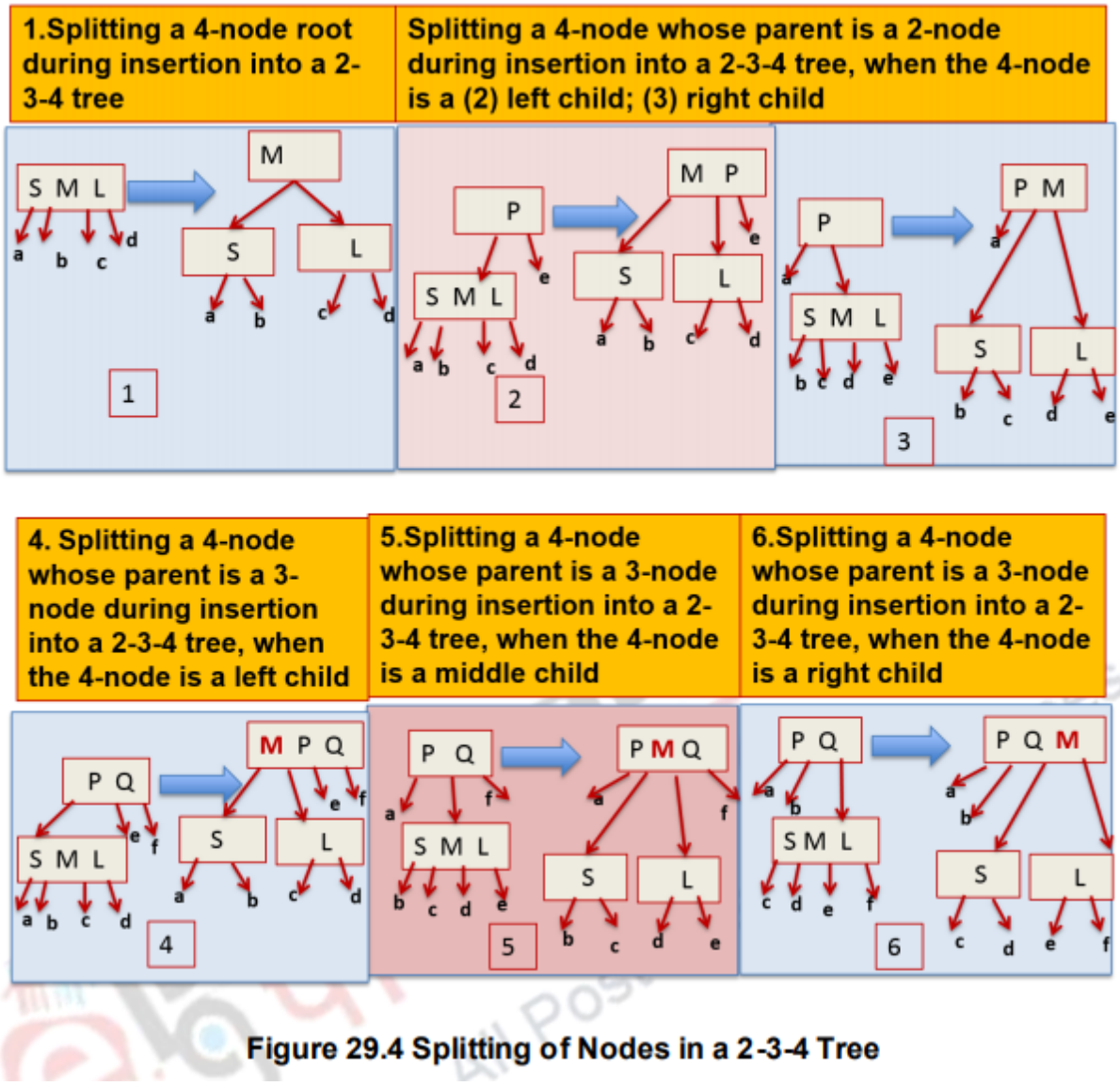


Figure 29.4 Splitting of Nodes in a 2-3-4 Tree

two subtrees with data values S and L each with two children respectively. These two newly formed subtrees are connected to a parent node with data value M.

The next two cases (Figure 29.4 (2) & (3)) we will discuss are the cases when the 4-node has a parent which is a 2-node and hence can accommodate the data value of the 4-node that is percolated up during its split. The parent node then becomes a 3-node. Now let us discuss the splitting of a 4-node whose parent is a 2-node during insertion into a 2-3-4 tree, when the 4-node is a left child (Figure 29.4 (2)). Here again we have the 4-node having data values S, M and L with children a, b, c, d. This 4-node node is connected to the parent node having data value P as its left child with the parent node's other child being e (that is the parent node is a 2-node). As in the general case we split the 4-node into two subtrees with data values S and

L each with two children a, b and c, d respectively. These newly formed two subtrees are connected to the 4-node's original parent with data value M being added to the left of P of the parent node. The parent node now becomes a 3-node. Similarly we will now discuss the splitting of a 4-node whose parent is a 2-node during insertion into a 2-3-4 tree, when the 4-node is a right child (Figure 29.4 (3)). Here we have the 4-node having data values S, M and L with children b, c, d, e. This 4-node node is connected to the parent node having data value P as its right child with the parent node's left child being a (that is the parent node is a 2-node). As in the general case we split the 4-node into two subtrees with data values S and L each with two children b, c and d, e respectively. These newly formed two subtrees are connected to the 4-node's original parent with data value M being added to the right of P of the parent node. The parent node now becomes a 3-node.

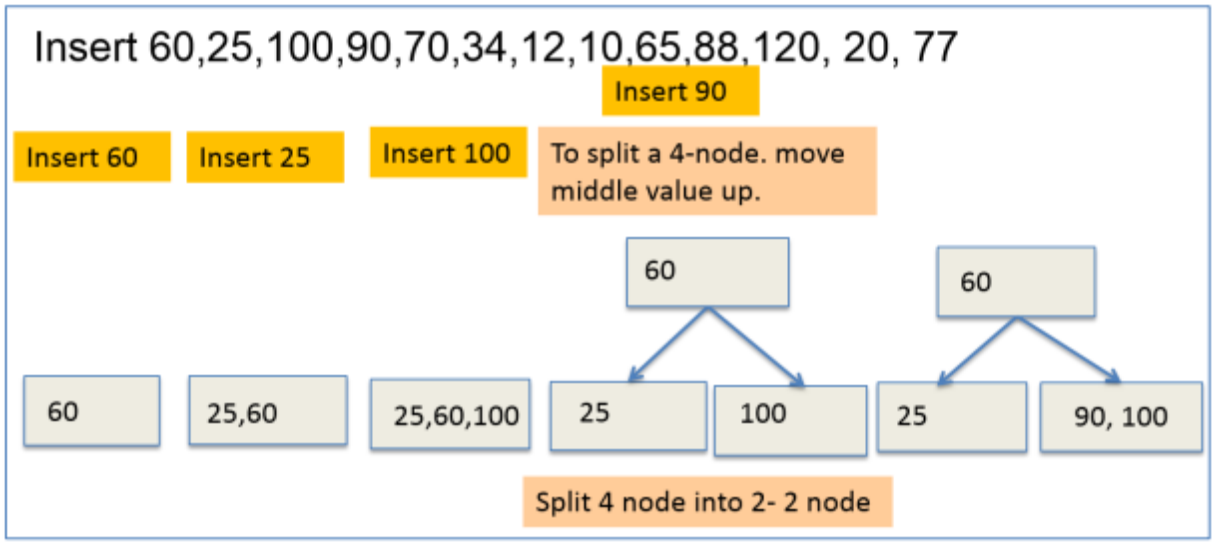
The next three cases (Figure 29.4 (4), (5) & (6)) we will discuss are the cases when the 4-node has a parent which is a 3-node and hence can accommodate the data value of the 4-node that is percolated up during its split. The parent node then becomes a 4-node. Now let us discuss the splitting of a 4-node whose parent is a 3-node during insertion into a 2-3-4 tree, when the 4-node is a left child (Figure 29.4

(4)). Here again we have the 4-node having data values S, M and L with children a, b, c, d. This 4-node node is connected to the parent node having data value P and Q as its left child with the parent node's two children being e and f (that is the parent node is a 3-node). As in the general case we split the 4-node into two subtrees with data values S and L each with two children a, b and c, d respectively. These newly formed two subtrees are connected to the 4-node's original parent with data value M being added to the left of P and Q of the parent node. The parent node now becomes a 3-node. Similarly we can understand the splitting a 4-node whose parent is a 3-node during insertion into a 2-3-4 tree, when the 4-node is a middle child where the two newly formed children of the 4-node become the middle two children of the modified 4-node parent (Figure 29.4 (5)). Similar is the case of splitting a 4-node when the 4-node is a right child where the two newly formed children formed due to the splitting of the 4-node become the right most two children of the modified 4-node parent (Figure 29.4 (6)).

29.3.2 Example of Insertion

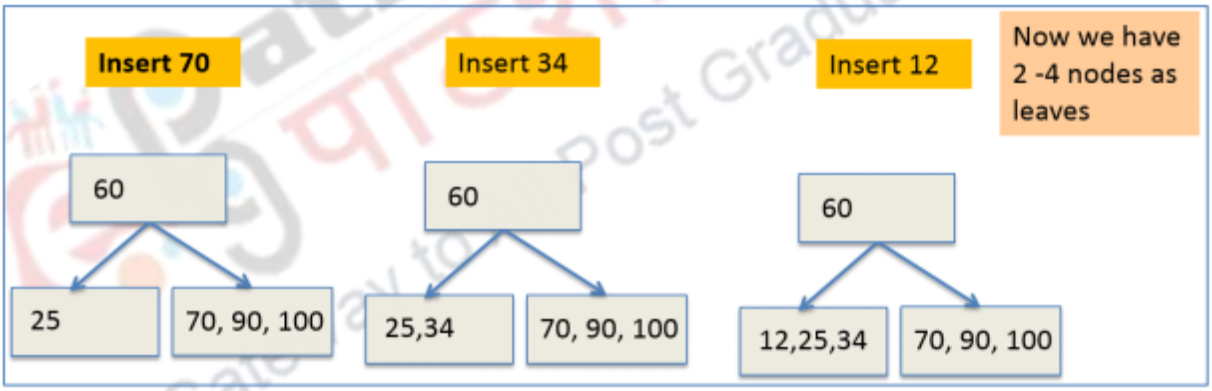
Now let us understand the insertion and the splitting using an example. Let us

the case where we insert **60,25,100,90,70,34,12,10,65,88,120, 20, 77**. Initially 60 is inserted as the only node. Next 25 is added as a data value to the node to the left of 60 (since 25<60). Now we add 100 as a data value to the node to the right of 60 (since 100>60). Now we have a 2-3-4 tree with 3 data values in the one 4-node it contains. Now we want to insert the data item 90. We need to split the 4-node into 2-2 nodes before insertion can be done. Therefore the node is split into 2-2 nodes with data values 25 and 60 respectively which become the left and right children of a newly created parent node which has the middle value (60) of the original node as its data value. Next 90 can be inserted into the right subtree of parent as a data item to the left of 100. (Figure 29.5 (a)).



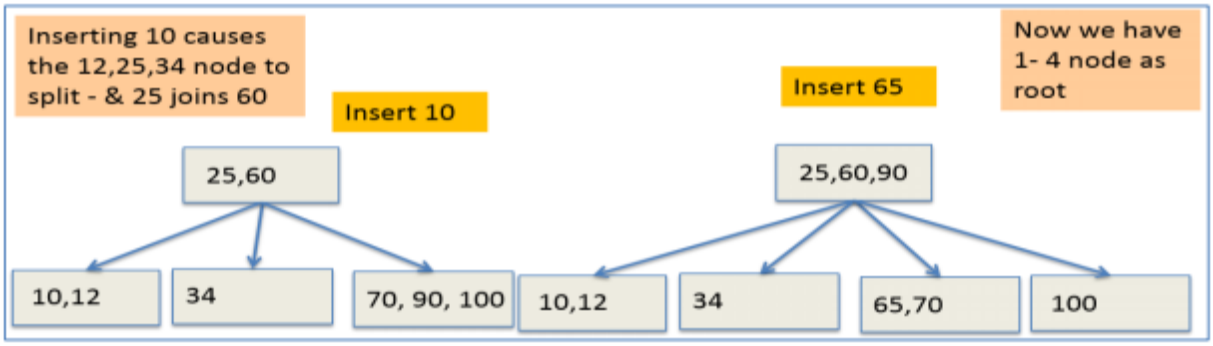
(a)

Now we continue to insert data values 70 and 34. 70 goes as the third child (left data item) of the right leaf, making this right leaf a 4-node. We split leaf nodes only when insertion causes the data values to increase beyond three. 34 goes as the second child (right data item) of the left leaf. Now we insert 12. 12 goes as the third child (left data item) of the left leaf, making this left leaf a 4-node. Now we have two 4-nodes as the leaves (Figure 29.5 (b)).



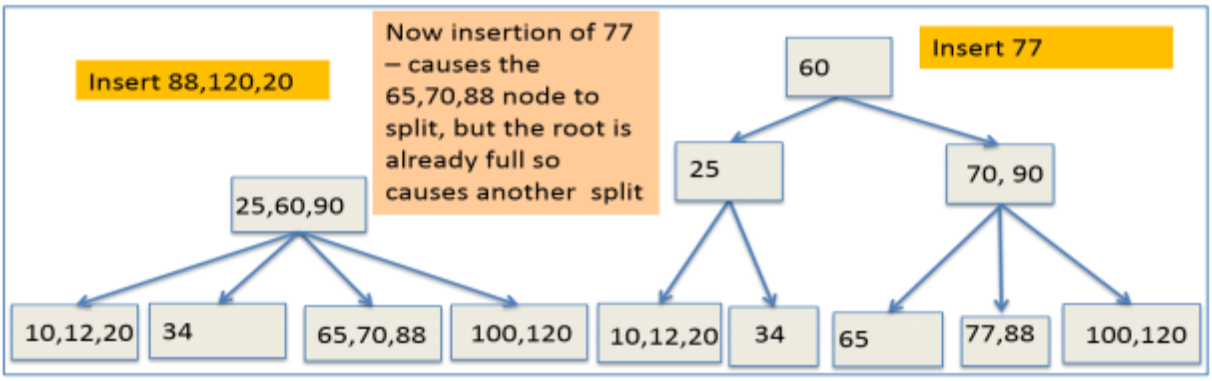
(b)

Next we insert 10. Inserting 10 causes 12-25-34 4-node leaf to split and 25 percolates to the parent. Next when we add 65, the 70-90-100 4-node leaf node splits causing 1 -4node to be formed at the root (Figure 29.5 (c)).



(c)

Now when we next add 88,120 and 20 the leaves get filled up. Now when we add 77, this causes the 65-70-88 4-node to split pushing up 70, but the root is already full so this causes another split where 60 gets pushed one more level up while 25 becomes data value of a 2-node left child of this root while 70 and 90 become data values of its 3-node right child (Figure 29.5 (d)).



(d)

Figure 29.5 Example of Insertion

The important part of the insertion algorithm is how to split a 4-node. The key aspect for this split is that as you walk down the tree a parent will always have room for a value. If node is a 4-node, split the 3 values into a left 2-node, a right 2-node, and promote the middle element to the parent of the node (which definitely has room) attaching children appropriately.

29.4 Deletion from 2-3-4 Tree

Here the deletion is similar to deletion from a binary search tree. We find the value to be deleted and swap with this with the inorder successor. In 2-3-4 trees the inorder successor is defined as traversing left, middle, right order. On the way down the tree (both for value and successor), we upgrade 2-nodes into 3-nodes or 4 nodes. This ensures that the deleted value will be in a 3-node or 4-node leaf. We then remove the value from the leaf.

For the deletion procedure the items are deleted at the leafs à swap item of internal node with inorder successor. However removal of a data value from a 2-node leaf creates a problem. Many different strategies are possible. One way is on the way from the root down to the leaf turn 2-nodes (except root) into 3 -nodes. This procedure is called upgrading. Make the following adjustments when a 2-node – except the root node – is encountered on the way to the leaf we want to remove

The deletion of an arbitrary element from a 2-3-4 tree may be reduced to that of

a deletion of an element that is in a leaf node.

If the element to be deleted is in a leaf that is a 3 -node or 4-node, the its deletion leaves behind a 2-node or a 3-node. No restructure is required.
To avoid a backward restructuring path, it is necessary to ensure that at the time of deletion, the element to be deleted is in a 3-node or a 4-node. This is accomplished by restructuring the 2-3-4 tree during the downward pass.

Suppose the search is presently at node p and will move next to node q. The following cases need to be considered:

- p is a leaf: The element to be deleted is either in p or not in the tree.
- q is not a 2-node. In this case, the search moves to q, and no restructuring is needed.
- q is a 2-node, and its nearest sibling, r, is also a 2 -node.
- if p is a 2-node, p must be root, and p, q, r are combined by reserving the 4-node being the root splitting transformation.
- if p is a 3-node or a 4-node, perform, in reverse, the 4 -node splitting transformation.
- q is a 2-node, and its nearest sibling, r, is a 3-node. is a 2-node and its nearest sibling, r, is a 4-node.

29.4.1 Upgrade Cases

- **2-node whose next sibling is a 2-node**

In this case we combine sibling values and “divider” value from parent into a 4-node. By the algorithm, parent cannot be a 2-node unless it is the root; in this case, our new 4-node becomes the root

- **2-node whose next sibling is a 3-node**

In this case we move this value up to parent, move divider value down, and shift a value to sibling.

29.4.2 Turning a 2-node into a 3-node ...

:Case 1: an adjacent sibling has 2 or 3 items

If a sibling on either side of this node is a 3-node or a 4-node (thus having more than 1 key), perform a rotation with that sibling where t he key from the other sibling closest to this node moves up to the parent key that overlooks the two nodes. The parent key moves down to this node to form a 3-node. The child that was originally with the rotated sibling key is now this node's additional child.

Here we "steal" item from sibling (10-20 Node) for converting 2-node 40 by rotating items and moving subtree . Here we left rotate 10-20 3-node about the 30-50 3-node and the 40 2-node becomes a 30-40 3 node (Figure 29.6).

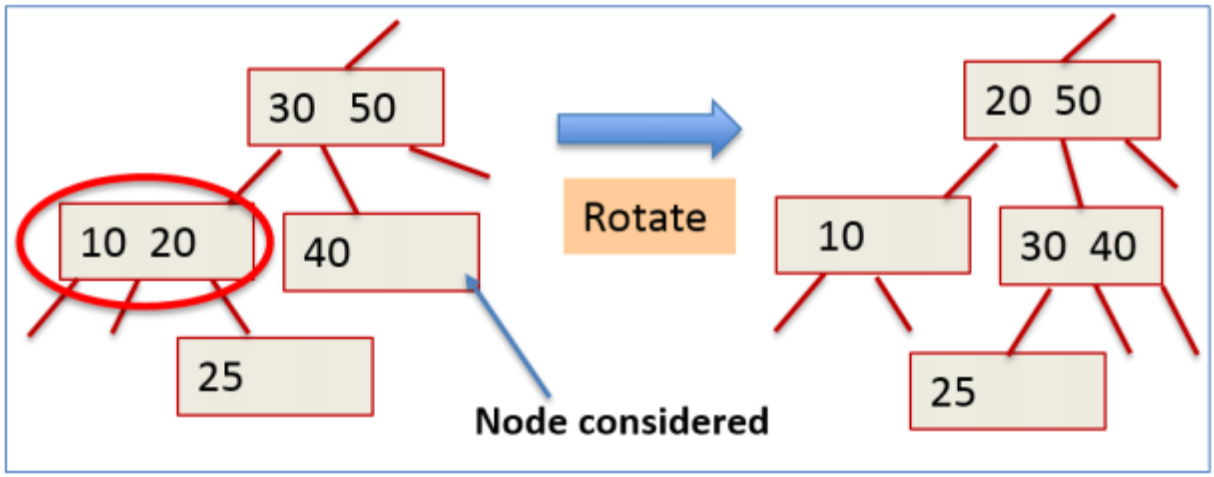


Figure 29.6 Adjacent Sibling has 2 or 3 items

Case 2: Each adjacent sibling has only one item

If the parent is a 2-node and the sibling is also a 2-node, combine all three elements to form a new 4-node and shorten the tree. (This rule can only trigger if the parent 2-node is the root, since all other 2-nodes along the way will have been modified to not be 2-nodes. This is why "shorten the tree" here preserves balance; this is also an important assumption for the fusion operation.) If the parent is a 3-node or a 4-node and all adjacent siblings are 2-nodes, do a fusion operation with the parent and an adjacent sibling, the adjacent sibling and the parent key

overlooking the two sibling nodes come together to form a 4-node. Then we transfer the sibling's children to this node.

We want to merge the two 2-node siblings 10 and 40. Here we "steal" item (in this case 40) from parent and merge node with sibling. Here the parent node has at least two items. Here we right rotate 40 about the 30-50 node so that 10-30-40 becomes a merged node (Figure 29.7).

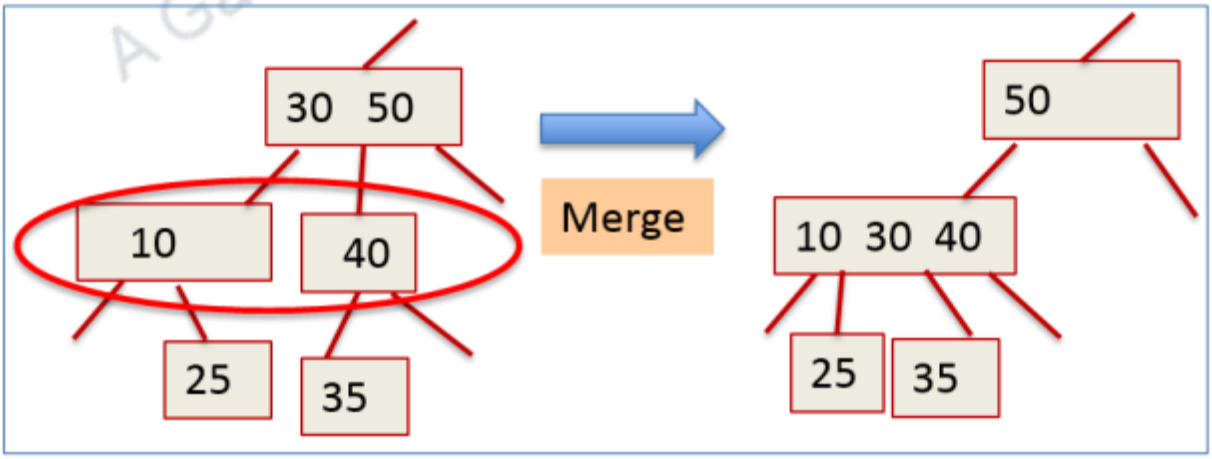


Figure 29.7 Adjacent Siblings have only one item each

29.5 Efficiency of 2-3-4 Tree

Here we are guaranteed O(log N) time for search and insert. This is because when searching for an item in a 2-3-4 tree with n elements, the maximum number of nodes visited during the search is int(log2n) + 1. Inserting an element into a 2-3-4 tree requires splitting no more than int(log2n) + 1 4-nodes. Normally requires only far fewer splits

29.6 Drawbacks of 2-3-4 Trees

The main disadvantage is the awkwardness to maintain three types of nodes. The standard search of binary trees need to be modified. Splits need to move links between nodes. The coding is complex since there is a need to handle many cases. Since any node may become a 4-node, all nodes must have space for 3 values and 4 links, however most nodes are not 4-nodes. This results in lots of wasted memory,

unless implementation is more specialized. Due to complex nodes and links, 2-3-4 operations are slower than binary search trees .

Summary

- Explained the concept of 2-3-4 Trees
- Outlined insertion into 2 -3-4 Trees
- Described deletion from 2 -3-4 Trees
- Discussed the advantages and disadvantages of 2-3-4 Trees

---

you can view video on 2-3-4 Trees 

Web Links
http://www.cs.mcgill.ca/~cs251/ClosestPair/2-4trees.html
https://www.cs.usfca.edu/~galles/visualization/BTree.html
http://www.serc.iisc.ernet.in/~viren/Courses/2010/SE286/Lecture16.pdf
https://www.cs.purdue.edu/homes/ayg/CS251/slides/chap13a.pdf https://www.cs.purdue.edu/homes/ayg/CS251/slides/chap13b.pdf
http://www2.thu.edu.tw/~emtools/Adv.%20Data%20Structure/2-3,2-3-4%26red-blackTree_952.pdf
https://cw.fel.cvut.cz/wiki/_media/courses/a4m33pal/paska12x.pdf
Supporting & Reference Materials
Mark Allen Weiss, “Data Structures and Algorithm Analysis in Java”, Pearson; 3rd Edition, 2011
Carrano and Henry, “Data Structures and Problem Solving with C++: Walls and Mirrors”, Pearson; 6 edition, 2012
Adam Drozdek, “Data Structures and Algorithms in C++”, Cengage; 4th edition,2013
Alfred V. Aho and Jeffrey D. Ullman, “Data Structures and Algorithms”, 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, “Data Structures and Algorithms in Java”, Wiley; 6 edition, 2014